

Homework Assignment #3
Due: September 25, 2024, by 11:59 pm

- **You must submit your assignment through the Crowdmark system.** You will receive by email an invitation through which you can submit your work in the form of separate PDF documents with your answers to each question of the assignment. To work with a partner, you and your partner must form a group on Crowdmark. Crowdmark does not enforce a limit on the size of groups. **The course policy that limits the size of each group to at most two remains in effect:** submissions by groups of more than two persons will not be graded.
- To minimize bias in grading, Crowdmark does not reveal your name to the grader. Please do not subvert this feature by including your name(s) on the files you submit.
- It is your responsibility to ensure that the PDF files you submit are legible. To this end, I encourage you to learn and use the LaTeX typesetting system, which is designed to produce high-quality documents that contain mathematical notation. You are not required to produce the PDF files you submit using LaTeX; you may produce it any way you wish, as long as the resulting document is legible.
- By virtue of submitting this assignment you (and your partner, if you have one) acknowledge that you are aware of the policy on homework collaboration for this course.^a
- For any question, you may use data structures and algorithms previously described in class, or in prerequisites of this course, without describing them. You may also use any result that we covered in class, or is in the assigned sections of the official course textbooks, by referring to it.
- Unless we explicitly state otherwise, you may describe algorithms in high-level pseudocode or point-form English, whichever leads to a clearer and simpler description. Do not provide executable code.
- Unless we explicitly state otherwise, you should justify your answers. Your paper will be graded based on **the correctness and efficiency of your answers, and the clarity, precision, and conciseness of your presentation.**

^a“In each homework assignment you may collaborate with at most one other student who is currently taking CSCC73. If you collaborate with another student on an assignment, you and your partner must submit only one copy of your solution, with both of your names. The solution will be graded in the usual way and both partners will receive the same mark. Collaboration involving more than two students is not allowed. **For help with your homework you may consult only the instructor, TAs, your homework partner (if you have one), your textbook, and your class notes. You may not consult any other source.**”

Question 1. (20 marks) Consider the algorithm LOOPYSORT(A, ℓ, r) below, where $A[1..n]$ is an array, and $1 \leq \ell \leq r \leq n$. This is a recursive sorting algorithm that (unlike Mergesort) requires no merging and (unlike Quicksort) requires no partitioning.

```
LOOPYSORT( $A, \ell, r$ )
1  if  $r = \ell$  then                                ▷  $A[\ell..r]$  has length 1
2      return
3  elseif  $r = \ell + 1$  then                          ▷  $A[\ell..r]$  has length 2
4      if  $A[\ell] > A[r]$  then swap  $A[\ell]$  and  $A[r]$ 
5      return
6  else                                              ▷  $A[\ell..r]$  has length at least 3
7       $t := \lceil 2(r - \ell + 1)/3 \rceil$                 ▷  $t$  is two-thirds of the length of  $A[\ell..r]$ 
8      LOOPYSORT( $A, \ell, \ell + t - 1$ )                ▷ sort the first two-thirds of  $A[\ell..r]$ 
9      LOOPYSORT( $A, r - t + 1, r$ )                    ▷ sort the last two-thirds of  $A[\ell..r]$ 
10     LOOPYSORT( $A, \ell, \ell + t - 1$ )                ▷ re-sort the first two-thirds of  $A[\ell..r]$ 
11     return
```

- a. Prove that a call to $\text{LOOPYSORT}(A, \ell, r)$ sorts the subarray $A[\ell..r]$ in increasing order. To simplify matters, you may assume that the length $r - \ell + 1$ of $A[\ell..r]$ is a power of 3.
- b. Use the Master Theorem to determine the running time $T(n)$ of the call $\text{LOOPYSORT}(A, 1, n)$.
- c. How does the running time of $\text{LOOPYSORT}(A, 1, n)$ compare to the running time of Bubblesort on array $A[1..n]$? Justify your answer.

Question 2. (20 marks) Let $A[1..n]$ be an array about which the following are true:

- (a) The positions of A contain some number k of distinct elements.
- (b) One of these k elements appears in exactly one position of A ; we will call this unique position *singleton*.
- (c) Each of the remaining $k - 1$ elements appears in exactly *two consecutive positions* of A ; we will call a pair of consecutive positions containing the same element a *couple*.

We want to design an algorithm that, given the array A that satisfies the above properties, returns the singleton (i.e., the position of the unique element).

There is a straightforward $\Theta(n)$ algorithm that does this. Give an asymptotically faster divide-and-conquer algorithm that solves this problem.¹ Justify the correctness of your algorithm, and analyze its running time using the Master Theorem.

Hint: What is n in terms of k ? Is n even, odd, or could it be either?

¹An algorithm with running time $\Theta(f(n))$ is *asymptotically faster* than an algorithm with running time $\Theta(g(n))$ if $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$; this is denoted $f(n) = o(g(n))$ or $f(n) \in o(g(n))$, and is read as “ $f(n)$ is little-oh of $g(n)$ ”. Thus, an algorithm that runs in $\Theta(\log n)$ time is asymptotically faster than one that runs in $\Theta(\sqrt{n})$ time, which is asymptotically faster than one that runs in $\Theta(n)$ time, which is asymptotically faster than one that runs in $\Theta(n \log n)$ time.